

第 10 章 附录：命令速查与常见问题

10.1 命令速查表

一次性配置（每台电脑只做一次）

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
ssh-keygen -t ed25519 -C "你的邮箱" # 生成 SSH 密钥（公钥添加到 GitHub）
```

新项目上传（第 4 章）

```
git init # 初始化
git add . # 暂存所有文件
git commit -m "首次提交" # 提交
git branch -M main # 分支改名 main
git remote add origin git@github.com:用户名/仓库名.git
git push -u origin main # 推送（-u 后日常只需 git push）
```

日常循环（第 5 章）

```
git pull # 开工拉取
git status # 看状态
git diff # 看具体改动
git add . # 暂存
git commit -m "类型：说明" # 提交
git push # 推送
git log --oneline --graph # 看历史
```

后悔药（第 6 章）

```
git restore --staged 文件 # 取消暂存
git restore 文件 # 丢弃工作区改动 ⚠️
git commit --amend -m "新信息" # 改最近提交信息
git reset --soft HEAD~1 # 撤销最近提交，保留改动
git reset --hard 提交ID # 回到历史版本 ⚠️
git revert 提交ID # 反向抵消已推送的提交
```

分支 (第 7 章)

```
git branch                # 查看分支
git switch 分支名         # 切换
git switch -c 分支名      # 创建并切换
git merge 分支名          # 合并到当前分支
git branch -d 分支名      # 删除分支
git push origin --delete 分支名 # 删除远程分支
git merge --abort         # 取消合并（冲突时）
```

其他常用

```
git stash                # 暂存改动（切走干别的）
git stash pop            # 取回暂存的改动
git remote -v            # 查看远程地址
git clone git@github.com:用户名/仓库名.git # 克隆仓库
git tag v1.0.0 && git push --tags # 打版本标签
```

10.2 常见报错对照表

报错信息	原因	解决办法
<code>fatal: not a git repository</code>	在非仓库文件夹里执行了 Git 命令	先 <code>cd</code> 到项目文件夹，或右键文件夹选 Git Bash Here
<code>fatal: unable to access ...</code> <code>Failed to connect</code>	网络问题 / 代理问题	检查网络；公司网络可能屏蔽，尝试手机热点测试；有代理时检查 <code>git config --global http.proxy</code>
<code>Permission denied</code> <code>(publickey)</code>	SSH 密钥没配好	按第 2 章 2.4 节重配；或改用 HTTPS 地址 + Personal Access Token
<code>! [rejected] main -> main</code> <code>(fetch first)</code>	云端有本地没有的提交	<code>git pull --rebase</code> 后再 <code>push</code> ；有冲突先解决
<code>! [rejected] main -> main</code> <code>(non-fast-forward)</code>	同上，历史分叉了	同上；单人仓库可 <code>git pull</code> 后合并再 <code>push</code>
<code>error: remote origin already exists</code>	重复添加远程地址	忽略它，直接 <code>push</code> ；或 <code>git remote set-url origin</code> 新地址
<code>Please tell me who you are</code>	没配置用户名邮箱	执行 2.3 节的两条 <code>config</code> 命令
<code>Everything up-to-date</code>	没有新东西要推	不是错误，云端已是最新
<code>warning: LF will be replaced by CRLF</code>	Windows 换行符警告	无碍；执行 <code>git config --global core.autocrlf true</code> 消除
提交后陷入奇怪的编辑界面	<code>commit</code> 忘了加 <code>-m</code> ，进入了 Vim	按 <code>Esc</code> ，输入 <code>:q!</code> 回车退出，重新带 <code>-m</code> 提交

报错信息	原因	解决办法
Your branch is ahead of origin/main by 1 commit	本地有提交没推送	执行 <code>git push</code> 即可
Support for password authentication was removed	用 HTTPS + 密码推送	GitHub 已禁用密码，改用 SSH（推荐）或 Personal Access Token

10.3 .gitignore: 让 Git 忽略该忽略的文件

有些文件**永远不该上传**:

- 依赖包目录 (`node_modules/`、`__pycache__`) —— 体积巨大且可重新安装
- 密钥文件 (`.env`、`*.key`) —— **泄露等于裸奔**，这是安全红线
- 系统垃圾 (`Thumbs.db`、`.DS_Store`)、IDE 配置 (`.vscode/`、`.idea/`)

用法: 在仓库根目录创建 `.gitignore` 文件，每行一条规则:

```
# 注释用 # 开头

# 依赖目录
node_modules/
__pycache__/

# 环境变量和密钥（千万不要提交！）
.env
*.key

# 系统垃圾
.DS_Store
Thumbs.db

# 编辑器配置
.vscode/
.idea/
```

创建时机: **最好在第一次 `git add .` 之前**。如果已经提交过再添加 `.gitignore`，需要先 `git rm -r --cached` 要忽略的目录 清除跟踪记录。

已有现成模板: 建仓库时在 Add `.gitignore` 下拉框里选择你的语言 (Python、Node 等)，GitHub 自动生成一套合适的规则。

10.4 敏感信息泄露怎么办

如果你已经把密钥、密码提交并推送到了云端——**光删除文件远远不够，历史记录里还有**。正确做法:

1. **立即作废该密钥**: 去对应平台 (如阿里云、AWS) 吊销并重新生成，这是第一优先级

- 2. 如果仓库是私有的且没有别人看过：最简单是**删除整个仓库重建**（把本地敏感文件清掉后重新推）
- 3. 如果是公开仓库：用 `git filter-repo` 清洗全部历史，并知会协作者

原则：**默认「一旦推到云端就视为泄露」**。宁可本地多检查一步 `git status`，也不要赌「私有仓库应该没人看」。

10.5 HTTPS 方式的备用方案（SSH 配不好时用）

公司网络有时屏蔽 SSH 的 22 端口，可以退回 HTTPS：

```
git clone https://github.com/用户名/仓库名.git
```

但 HTTPS 推送需要「密码」——现在要用 **Personal Access Token（个人访问令牌）** 代替：

- 1. GitHub → 头像 → Settings → 最底部 **Developer settings**
- 2. **Personal access tokens** → **Tokens (classic)** → **Generate new token**
- 3. 勾选 `repo` 权限，生成后**立即复制保存**（只显示一次）
- 4. 推送时用户名填 GitHub 用户名，密码粘贴这个 token

10.6 进阶学习资源

资源	地址	说明
GitHub 官方文档	https://docs.github.com	最权威，有中文版
交互式 Git 分支练习	https://learngitbranching.js.org	图形化闯关，强烈推荐
Pro Git 中文版	https://git-scm.com/book/zh/v2	系统学习 Git 的经典教材
GitHub Skills	https://skills.github.com	GitHub 官方的交互式入门课
Oh Shit, Git!?!	https://ohshitgit.com	各种「搞砸了怎么办」的急救手册

10.7 最后的建议

- 1. **从小开始**：先一个人用 Git 一个月，再谈协作；先用好 `add/commit/push`，再学分支
- 2. **提交要勤**：宁可一天提交十次小改动，不要一周提交一次大杂烩
- 3. **push 前先 status + diff**：养成自查习惯，能避免 90% 的事故
- 4. **别怕搞砸**：本地仓库的一切都有后悔药（第 6 章）；真正不可逆的只有「把密钥推到公开仓库」
- 5. **多用 GitHub 搜**：你遇到的几乎所有问题，别人都在 Issues 里问过、解决过

恭喜读完！现在打开 Git Bash，敲下 `git status`，开始你的第一个项目吧。